Stubbing Behavior

In Mockito Architecture topic, I have already covered how stubbing works internally

By default, Mockito mocks:

- return null for objects
- return 0 for primitives
- return false for boolean
- return empty collections (for some types)

But Stubbing overrides that.

For example:

In below test case, I have not used any stubbing.

```
@Test

void testWithoutStub() {

Calculator calculator = Mockito.mock(Calculator.class);
int sum = calculator.add(4, 2);

assertEquals(6, sum);

}

```

```
12
```

Lets see different ways of stubbing:

1. **when(...).thenReturn(...)**

Defines the behavior to return a specific value when a certain condition is met.

```
@Test
void testFixedReturn1() {
    Calculator calculator = Mockito.mock(Calculator.class);
   ` /*
    This does exact argument matching
    */

    // stubbing
    Mockito.when(calculator.add(4, 2)).thenReturn(10);

    int sum = calculator.add(4, 2);
    assertEquals(10, sum);
}
```

Means, if this "add()" method get invoked with other parameters like: calculator.add(5,4);

Then this mock stub "*Mockito.when(calculator.add(4, 2)).thenReturn(10)*" will not be used, as arguments are not matched.

So, calculator.add(5,4) will return default value i.e 0.

2. **when(...).thenThrow(...)**

Is used to stub a method so that it throws a specified exception when invoked during a test.

There are 2 types of exceptions:-

1. Unchecked Exception
2. Checked Exception

1. **In Unchecked Exception:-**

- Runtime exception.

- Compiler do not enforce us to handle it.

```
public int add(int a, int b) {
        return a+b;
}

```

```
@Test
void testFixedReturn2() {

Calculator calculator = Mockito.mock(Calculator.class);

// stubbing
when(calculator.add(4, 2)).thenThrow(new NullPointerException());

assertThrows(NullPointerException.class, () -> calculator.add(4, 2));
}

```

This test case works fine

2. **In Checked Exception:-**

- Compile time exception.
- Compiler force us to handle it either by try/catch or Propagate it.


```
public int add(int a, int b) throws IOException {
        return a+b;
}

```

Test method should match the method signature

```
@Test
void testFixedReturn2() throws IOException {

Calculator calculator = Mockito.mock(Calculator.class);

// stubbing
when(calculator.add(4, 2)).thenThrow(new IOException());

assertThrows(IOException.class, () -> calculator.add(4, 2));
```

```
}
```

3. **When(...).thenReturn(...).thenReturn(...)**

Is used for sequential stubbing, returning different values on successive invocations of the same method.

```
when(service.get("u123"))

     .thenReturn(null)

     .thenReturn("shrayansh");
```

Behavior:

```
1st call of service.get("u123")  ->  null

2nd call of service.get("u123") ->  shrayansh

3rd call of service.get("u123")  ->  shrayansh (last value repeats)
```

```
@Test
void testSequentialStub() {
        Calculator calculator = Mockito.mock(Calculator.class);
        // stubbing
        when(calculator.add(4, 2))
                    .thenReturn(5)
                    .thenReturn(6)
                    .thenThrow(new NullPointerException())
                    .thenReturn(7);

        assertEquals(5, calculator.add(4, 2));
        assertEquals(6, calculator.add(4, 2));
        assertThrows(NullPointerException.class, () ->
calculator.add(4, 2));
        assertEquals(7, calculator.add(4, 2));
}
```

4. **doReturn(...).when(...)**

- Stubs a method without invoking the real method during stubbing.
- Useful for spies

```
class Calculator {
```

```
    int add(int a, int b) {
        return a + b;
    }
}

```

```
Calculator calculator = spy(new Calculator());

// stubbing: But also invokes real method
    /*
      Like this, here stubbing is happening.
      But it will also invoke the multiply method
    */
when(calculator.add(4, 2)).thenReturn(100);

int sum = calculator.add(4, 2);       // returns 100, but it will also invoke
real method too

```

Safer version for spy objects:

Safe implemenation, make use of : doReturn for stubbing, to avoid calling the real method.

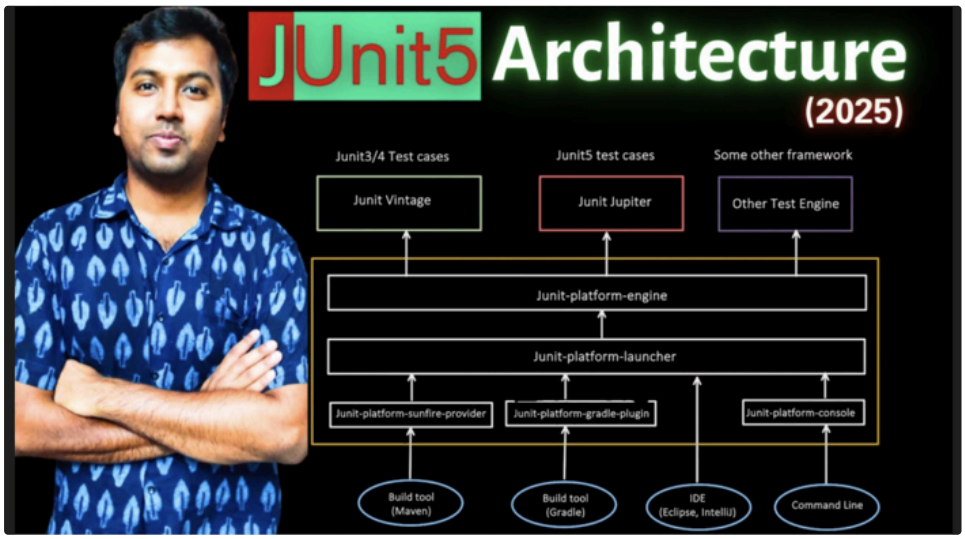
```
Calculator calculator = spy(new Calculator());

// stubbing: do not invoke real method, safe to use with spy
doReturn(100).when(calculator).add(4, 2);

int sum = calculator.add(4, 2);   // mocked → 100

```

Why doReturn().when() is safe for spy objects, I have already covered in Mockito Architecture video



5. **doNothing()**

Used to explicitly stub a void method, so that it performs no action when invoked.

```
class NotificationService {

         void sendEmail(String userId) {
                  . . .
        }
}

```

Using Mock:-

```
@Test
void testVoidReturnMethod() {

        NotificationService mockNotifyObj =
                        Mockito.mock(NotificationService.class);

        mockNotifyObj.sendMail();
}

```

All good, mockNotifyObj.sendMail() does nothing.

Using Spy:-

```
@Test
void testVoidReturnMethod() {

        NotificationService spyNotifyObj =
                        Mockito.spy(new NotificationService());

        spyNotifyObj.sendMail();
}

```

The above code will invoke the sendMail() method.

So, how to fix for Spy?

```
@Test
void testVoidReturnMethod() {

        NotificationService spyNotifyObj =
                        Mockito.spy(new NotificationService());

        doNothing().when(spyNotifyObj).sendMail();
}

```

6. **doThrow()**

It is used to stub a void method so that it throws a specified exception when invoked, without calling the real method.

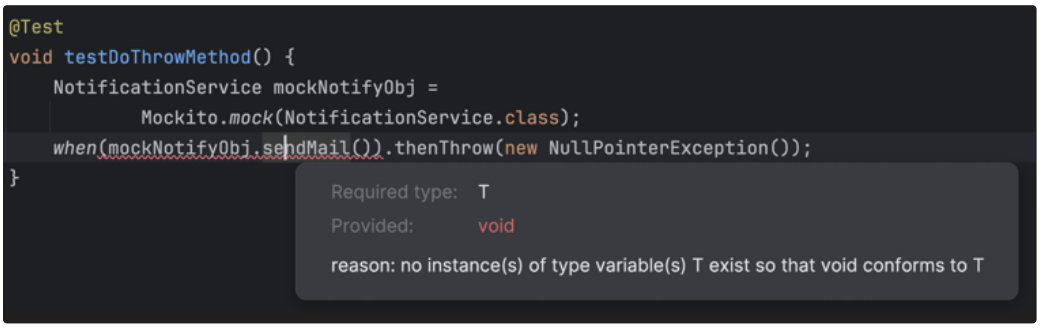
```
class NotificationService {

         void sendEmail(String userId) {
                  . . .
        }
}

```

How to test the Exception scenario for void method?

We can not use when(...).thenThrow(...)



Answer is: doThrow()

```
@Test
void testDoThrowMethod() {
        NotificationService mockNotifyObj =
                        Mockito.mock(NotificationService.class);

doThrow(new RuntimeException()).when(mockNotifyObj).sendMail();

    assertThrows(RuntimeException.class, () ->
mockNotifyObj.sendMail());
}

```

7. **Matchers**

Initially in start of this topic, we saw this example and the problem of *"exact argument matching"*

```
@Test
```

```
void testFixedReturn1() {
    Calculator calculator = Mockito.mock(Calculator.class);
    /*
    This does exact argument matching
    */
    // stubbing
    Mockito.when(calculator.add(4, 2)).thenReturn(10);

    int sum = calculator.add(4, 2);
    assertEquals(10, sum);
}

```

Means, if this "add()" method get invoked with other parameters like: **calculator.add(5,4);**

Then this mock stub **"*Mockito.when(calculator.add(4, 2)).thenReturn(10)*"** will not be used, as arguments are not matched.

So, **calculator.add(5,4)** will return default value i.e 0.

How we can resolve this? Answer is: **Matchers**

Mockito matchers allow flexible argument matching when stubbing method calls.

```
@Test
void testFixedReturn1() {
    Calculator calculator = Mockito.mock(Calculator.class);

    // stubbing
    Mockito.when(calculator.add(anyInt(), anyInt())).thenReturn(10);

    assertEquals(10, calculator.add(4, 2));
    assertEquals(10, calculator.add(5, 4));

}

```

Now the stub applies to *any* int arguments.

Matcher	Description	Example
any()	Matches any object of any type (including	when(service.process(any())).thenReturn("ok"

	null)	);
anyString()	Matches any String (including null)	when(service.getUser(anyString())).thenReturn(user);
anyInt()	Matches any int value	when(service.add(anyInt(), anyInt())).thenReturn(10);
anyLong()	Matches any long value	when(repo.findById(anyLong())).thenReturn(entity);
anyDouble()	Matches any double value	when(service.applyDiscount(anyDouble())).thenReturn(100.0);
anyList()	Matches any List(including null)	when(service.process(anyList())).thenReturn(true);
anyMap()	Matches any Map(including null)	when(service.process(anyMap())).thenReturn(true);
eq()	Matches an exact value (used when mixing matchers)	when(service.add(eq(5), anyInt())).thenReturn(15);

	If we use one matcher → all arguments must use matchers.	Important:
		Wrong: Mixing Raw value and Matcher do not work
		when(calculator.add(anyInt(), 5)).thenReturn(15);
		Correct: use eq()
		when(calculator.add(anyInt(), eq(5))).thenReturn(15);

isNull()	Matches only null values	when(service.process(isNull())).thenReturn("empty");
notNull()	Matches non-null values	when(service.process(notNull())).thenReturn("valid");
gt()	Matches values greater than specific value	when(service.calculate(gt(100))).thenReturn("high");

lt()	Matches values less than specific number	when(service.calculate(lt(10))).thenReturn("low");
geq()	Matches values greater than or equal to specific value	when(service.calculate(geq(50))).thenReturn("eligible");
leq()	Matches values less than or equal to specific value	when(service.calculate(leq(20))).thenReturn("small");
contains()	Matches string containing given substring	when(service.getUser(contains("admin"))).thenReturn(adminUser);
startwith()	Matches string starting with given prefix	when(service.getUser(startsWith("user_"))).thenReturn(user);
endsWith()	Matches string ending with given suffix	when(service.getUser(endsWith("@gmail.com"))).thenReturn(gmailUser);
matches()	Matches string using regex pattern	when(service.getUser(matches("user\\d+"))).thenReturn(user);

8. **Dynamic Stubbing: thenAnswer() and doAnswer()**

Problem:

```
public class UserService {

```

```
    private final UserRepository repository;

    public UserService(UserRepository repository) {
            this.repository = repository;
    }

    public void register(User user) {
            repository.save(user);
    }

    public User getUser(Long id) {
        return repository.findById(id);
    }
}

```

Without Dynamic Stubbing:

```
@Test
void testWithoutDynamicStubbing() {

    UserRepository repository = mock(UserRepository.class);
    UserService userService = new UserService(repository);

    User user1 = new User(1L, "A");
    User user2 = new User(2L, "B");

    // Stubbing
    doNothing().when(repository).save(user1));
    doNothing().when(repository).save(user2));

    userService.register(user1);
    userService.register(user2);

   // Stubbing
    when(repository.findById(1L)).thenReturn(user1);
    when(repository.findById(2L)).thenReturn(user2);

    User result1 = userService.getUser(1L);
    User result2 = userService.getUser(2L);

     // assertion
    assertEquals("A", result1.getName());
    assertEquals("B", result2.getName());
}

```

This test case works fine, but it has some weakness:

1. *Every new user → we need to add 2 new Stubs (one for save and one for findById)*

10 users → 20stubs

2. *Not a realistic behavior of repository:*

   If we observe this test case properly, it actually do not test the behavior.

Real behavior which we intend to test is:

- *During save, it store the data.*
- *During find, it fetch from that storage.*

But this test case does nothing.

So, I would say, when our dependency is:

STATELESS *→ Output just depend only on input, Static stubbing is fine.*

But when its:

STATEFUL *→ Output depend on previous call, Dynamic stubbing can help.*

Stateful like:

- *Cache : get() depend on put()*
- *Retry logic: say for first 3 calls retry happens, 4th call it should throw exception.*

Etc..

With Dynamic Stubbing:

```
@Test
void testWithDynamicStubbing() {

    UserRepository repository = mock(UserRepository.class);
    UserService userService = new UserService(repository);

    Map<Long, User> store = new HashMap<>();

    // simulate save
    doAnswer(invocation -> {
            User user = invocation.getArgument(0);
            store.put(user.getId(), user);
            return null;
        }).when(repository).save(any());


    // simulate findById
    when(repository.findById(anyLong()))
        .thenAnswer(invocation -> {
            Long id = invocation.getArgument(0);
            return store.get(id);
        });

    User user1 = new User(1L, "A");
    User user2 = new User(2L, "B");

```

```
27      userService.register(user1);
28      userService.register(user2);
29
30      User result1 = userService.getUser(1L);
31      User result2 = userService.getUser(2L);
32
33      assertEquals("A", result1.getName());
34      assertEquals("B", result2.getName());
35  }
36
```